

Agora Solutions Architect

Live Commerce Platform - Interview Q&A

This document contains 106 detailed interview questions and answers for the AI-Powered Live Commerce Platform candidate exercise. Each answer references the agoshop prototype architecture: ConvoAI + MCP, dual RTC channels, RTM chat sharding, SSE state sync, LIVE20 promotions, and Media Push CDN tier flip.

Minimum 280 words per answer. Generated from docs/interview-prep/build_qa.py

Q1. Which Agora capabilities would you use - and why?

PART 1 - Official Assignment Architecture Questions

In our prototype monorepo, Agora owns real-time media, signaling, and managed speech, while PostgreSQL remains the commerce source of truth. The Express Commerce API on port 8787 orchestrates sessions, cart, and ConvoAI join; the SSE gateway on 8789 fans out Redis pub/sub; the AI service on 8790 exposes streamable HTTP MCP at /mcp. nginx on 8080 routes by path with sticky MCP affinity on X-Convo-Id when scaled horizontally. For live commerce I would compose Agora RTC Interactive Live Streaming for the host-to-audience path, Agora RTM (Signaling REST on the server, RTM SDK in the browser) for sharded public chat and ConvoAI transcript delivery, Agora Conversational AI Engine for the private voice shopping assistant with managed ASR, LLM, and TTS, Media Push to originate an RTMP/HLS leg when viewer counts exceed the RTC interactive tier, Cloud Recording or browser MediaRecorder for replay assets, and Real-Time STT (RTT) when captions are required. RTC is chosen because shoppers need sub-second lip sync, barge-in friendly audio, and the ability to flip delivery tiers without abandoning the session. RTM is chosen because chat must fan out across frozen shard channels (chat-<slug>-<index>) without overloading a single channel or the 50 RTM subscriptions per client limit. ConvoAI is chosen because it collapses the voice loop - capture, understanding, tool planning, speech synthesis - into a managed pipeline that can call our MCP endpoint securely. Media Push is chosen because one million viewers cannot economically or technically subscribe as RTC audience on a single channel; the prototype proves the one-way rtc->cdn transition in deliveryTier.ts with Redis presence driving the threshold.

Each capability maps to a package under packages/agora (convoai.ts, mediapush.ts, recording.ts, rtt.ts, signaling.ts, tokens.ts) and domain logic under packages/domain-live and packages/domain-commerce. We deliberately keep business rules out of Agora Console except when AGORA_STUDIO_PIPELINE_ID overrides the programmatic prompt.

Q2. Would you use Agora Conversational AI Engine, Agora Studio, or another approach for the AI shopping assistant?

PART 1 - Official Assignment Architecture Questions

For the assignment prototype I use Agora Conversational AI Engine programmatically via `packages/agora/src/convoai.ts` and `packages/ai/src/conversations.ts`, not a standalone custom voice stack and not Studio-only configuration. Programmatic join lets us inject MCP headers (X-Convo-Id, X-Convo-Expires, X-Convo-Signature), set `remote_rtc_uids` to the shopper's freshly allocated UID, pin the agent to `ai-<conversationId>`, enable RTM `data_channel` for transcripts, and enforce `allowed_tools` against `MCP_ALLOWED_TOOLS`. Studio remains valuable for prompt iteration and pipeline tuning: when `AGORA_STUDIO_PIPELINE_ID` is set, Console agent configuration wins and our code-built prompt is suppressed - exactly how a Solutions Architect would split responsibilities between platform ops and application engineering.

An alternative build-your-own stack (Deepgram + Azure OpenAI + ElevenLabs + custom WebRTC) would maximize control but explode operational surface area and break the assignment's intent to show Agora-native voice commerce. ConvoAI gives us barge-in, turn detection, and tool orchestration aligned with Agora RTC/RTM. Our text fallback via `LLM_PROVIDER` on the Commerce API reuses the same `toolExecutor.ts` handlers when Redis `ai:agents` lease cap (`CONVOAI_MAX_CONCURRENT_AGENTS`, default 20) returns 503 `ai_capacity`.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

Q3. How would you design the host-to-audience media topology for the prototype and for a large-scale production deployment?

PART 1 - Official Assignment Architecture Questions

Prototype topology: the seller publishes from `useHostBroadcast.ts` into `live-<slug>` as RTC publisher; each viewer joins via `VideoStage.tsx` as audience level 2 on the same channel using a dedicated `IAgoraRTCCClient`. At go-live, `lifecycle.ts` starts side effects idempotently under `pg_advisory_xact_lock`: Cloud Recording or browser capture, RTT if enabled, and Media Push converter creation via `mediapush.ts` pushing to `MEDIA_PUSH_RTMP_URL/<slug>`. Redis presence in `presence.ts` tracks `session:<id>:viewers`; when pruned count reaches `RTC_TIER_MAX_VIEWERS` (3 in demo, thousands in production), `deliveryTier.ts` performs a conditional UPDATE from `rtc` to `cdn` exactly once and publishes `session.delivery_tier_changed` over SSE. Viewers then latch to HLS via `hls.js` while chat, polls, reactions, captions, and private ConvoAI remain real-time.

Production at viral scale adds CDN origin redundancy, multi-region Agora project routing, and likely co-host publishers with role-gated tokens. The host remains on RTC publisher; mass audience consumes CDN HLS or Agora Broadcast Streaming profile rather than holding interactive RTC subscriptions. Media Gateway (`mediagateway.ts`) supports OBS/RTMP ingest for professional sellers. The architecture preserves a single logical session with frozen `chatShardCount` and authoritative commerce state in Postgres regardless of delivery tier.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

Q4. Which capabilities belong in the customer application layer?

PART 1 - Official Assignment Architecture Questions

The customer application layer - Express Commerce API, domain-commerce, domain-live, SSE gateway, worker, and React web apps - owns everything that constitutes merchant truth and shopper entitlement. That includes catalog search, cart mutations, promotion evaluation (LIVE20 with requiresLiveSession), eligibility proofs in eligibility.ts, order creation, chat authorization and moderation, session lifecycle, ConvoAI conversation records, MCP tool execution, idempotency keys, and SSE event publication. Agora receives only tokens, channel names, and REST join payloads.

Concretely, when a shopper asks the voice agent for a live discount, ConvoAI plans a `get_live_offer` or `add_to_cart` tool call; the AI service at `/mcp` validates HMAC via `callbackAuth.ts`, runs `toolExecutor.ts` with deduplication on (`conversationId`, `toolCallId`), and returns structured JSON from Postgres. The browser never computes LIVE20 eligibility; it displays cart snapshots refetched after `cart.updated` SSE events. Redis holds ephemeral presence, `ai:agents` leases, moderation sets, and `promo:v1:active` cache - but not authoritative prices.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

Q5. How will the AI assistant securely access product information and invoke business APIs/tools?

PART 1 - Official Assignment Architecture Questions

Security is enforced at three layers: transport, authentication, and authorization inside tool handlers. Transport uses streamable HTTP MCP exposed only at `PUBLIC_API_URL/mcp`, reachable by Agora's managed LLM infrastructure. Authentication uses HMAC-SHA256 in `callbackAuth.ts`: `signCallback` produces `X-Convo-Signature` over `conversationId.expiresUnixSeconds` with `CONVO_LLM_SHARED_SECRET`; `mcp/auth.ts` validates on every request. Optional `MCP_STATIC_API_KEY` supports Console integration tests with Bearer plus `X-Convo-Id`. `nginx` sticky routes on `X-Convo-Id` keep a conversation pinned to one AI replica.

Authorization is tool-scoped: `joinConvoAiAgent` passes `allowed_tools` from `MCP_ALLOWED_TOOLS` - fourteen commerce tools including `search_products`, `get_live_offer`, `add_to_cart`, `escalate_to_human`. Mutating tools use claim/replay idempotency in `toolExecutor.ts` with `argsHash` to prevent duplicate cart lines on retries. The LLM never receives raw SQL or admin credentials; it receives tool schemas and structured responses. Catalog data never enters Agora prompts as unstructured dumps - we inject live promo rules via `systemPrompt.ts` but prices come back only through tool results.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

Q6. How will the application determine whether a live session is active and enforce the 20% live-session discount correctly?

PART 1 - Official Assignment Architecture Questions

LIVE20 is seeded with conditions.requiresLiveSession in seed.ts. Enforcement is server-side and relational, not client-side. resolveLineContext in packages/domain-commerce/src/eligibility.ts executes a single join across product_variants, products, live_sessions, and live_session_products. liveEligible is true only when liveSessionId is non-null, the session exists, session_status equals live, and the product is attached. evaluatePromotions in packages/shared/src/promotions.ts then rejects any promotion with requiresLiveSession when liveEligible is false.

When lifecycle.ts ends a session, it reprices all carts holding that liveSessionId and publishes cart.updated per user via Redis events:user. The AI cannot grant LIVE20 by narration alone: get_live_offer and add_to_cart both flow through the same resolver at execution time. Optional withSessionLiveRule in promotions.ts can inject a synthetic LIVE_SESSION promotion from liveSessions.discountPercent, but the attachment and live status proof remains identical. Admin can PATCH promotions and SSE promotions.changed triggers client refetch without redeploy.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

Q7. How will state be shared between the live experience, the AI conversation, and the shopping cart?

PART 1 - Official Assignment Architecture Questions

Shared state flows through Postgres as authority, Redis as hot mirrors and pub/sub bus, and SSE as the client notification layer - never through Agora channel metadata. The live page subscribes to session events on GET /api/events?sessionId= via useServerEvents.ts; cart state lives in React Query keys invalidated on cart.updated. When the voice agent adds an item, add_to_cart writes to Postgres, publishes to events:user, and the live UI refetches. Session status, delivery tier, viewer counts, RTT status, and chat moderation all use events:session or events:global.

The AI conversation context tool get_conversation_context reads the same liveSessionId and userId as the cart. ConvoAI transcripts arrive on RTM channels while commerce outcomes mutate Postgres. Audio ducking in useVoiceAgent.ts couples UX state locally (duck host track to 15% volume while agent speaks) without crossing server boundaries. On SSE reconnect, clients refetch snapshots rather than replay missed events - a deliberate choice for correctness over perfect event ordering.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

Q8. How would recorded sessions, transcription, privacy, consent, and retention be handled?

PART 1 - Official Assignment Architecture Questions

Recording in the prototype supports dual paths: browser MediaRecorder via useHostRecorder.ts for demo reliability, and Agora Cloud Recording behind RECORDING_PROVIDER=agora in recording.ts (acquire, mix mode, S3-compatible storage). RTT in rtt.ts provides Real-Time STT captions on live-<slug> with session.rtt_status_changed SSE events. ConvoAI transcripts flow via RTM data_channel, not durable voice storage by default.

Production privacy requires explicit consent banners before recording or AI voice processing, region-specific retention schedules, and separation of public stream recordings from private AI conversations. GDPR and India DPDP scenarios would disable voice retention entirely - transcripts processed ephemerally, MCP logs redacted, Cloud Recording lifecycle policies set to 30-90 days with legal hold exceptions. Support replay of private AI conversations would require separate consent and access controls; the prototype does not expose support playback of voice. PII in transcripts must be minimized at ingestion via tool design rather than LLM memory.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

Q9. What changes when the platform must support very high viewer concurrency and many simultaneous live sellers?

PART 1 - Official Assignment Architecture Questions

Vertical limits drive horizontal patterns already sketched in the prototype. One RTC channel cannot carry a million interactive subscribers - Media Push plus CDN tier flip in `deliveryTier.ts` is mandatory. RTM chat shards freeze at go-live: `chatShardCount = clamp(ceil(expectedPeakViewers / RTM_CHAT_SHARD_TARGET), 1, 49)` in `lifecycle.ts`, respecting Agora's 50 channels per RTM client. ConvoAI concurrency is capped by Redis sorted set `ai:agents` with worker lease sweep; beyond `CONVOAI_MAX_CONCURRENT_AGENTS`, voice degrades to text `LLM_PROVIDER` fallback.

Many simultaneous sellers multiply Postgres write load, Redis pub/sub fan-out, and SSE gateway connections - each becomes independently autoscaled. Signaling REST publish budget (500 req/s) forces batch fan-out in `signaling.ts` (25 channels per batch) and SSE-degraded chat fallback. Multi-tenant isolation uses per-session channels `live-<slug>`, per-conversation `ai-<conversationId>`, and per-user SSE channels. Operations add Agora Analytics, synthetic probes on `/mcp` health, and regional Agora projects for data residency.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

Q10. How would you monitor and troubleshoot failures across Agora, AI providers, and customer-owned services?

PART 1 - Official Assignment Architecture Questions

I would implement correlated observability across three planes: media (Agora RTC/RTM quality), voice AI (ConvoAI join, MCP latency, tool errors), and commerce (API 5xx, cart repricing, promotion evaluation). Structured logs already separate side service statuses - recordingStatus, rttStatus, mediaPushStatus - in lifecycle.ts with reconcileSessionEffects for healing. Dashboards would track ConvoAI 429/503 rates, MCP HMAC auth failures, ai:agents lease utilization, SSE gateway connect/disconnect churn, Redis publish lag, and delivery tier flip success.

Runbooks tie symptoms to subsystems: no agent audio -> verify ai-<conversationId> token role and remote_rtc_uids; chat silent -> check RTM REST budget or sse-degraded path; wrong discount -> trace resolveLineContext SQL result for attached and session_status; MCP failures -> validate PUBLIC_API_URL reachability and X-Convo-Signature expiry. Integration checks in packages/domain-live/src/__checks__ and packages/ai/src/mcp/__checks__ provide CI gates. For production I would add Agora Analytics call quality metrics, Sentry on API and AI service, and synthetic voice-turn probes that exercise search_products end-to-end every minute.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

Q11. A seller goes viral with 500K viewers - what breaks first?

PART 2 - Requirement Change Scenarios

At 500K concurrent viewers the first failure mode is economic and architectural, not a single bug: interactive RTC audience subscriptions on live-`<slug>` would overwhelm cost and Agora channel limits. The prototype's `RTC_TIER_MAX_VIEWERS` threshold exists precisely to trigger Media Push converter creation and the one-way `rtc->cdn` flip before that cliff. If Media Push or CDN origin is misconfigured, viewers still connect but video stalls while chat and AI continue - an confusing split-brain UX.

Second to break is RTM Signaling REST publish throughput: host fan-out to all chat shards batches 25 channels per request against a 500 req/s budget in `signaling.ts`; at extreme scale chat degrades to SSE transport with `chat.moderated degraded_chat` events. Third is ConvoAI: even with horizontal AI replicas, `CONVOAI_MAX_CONCURRENT_AGENTS` default 20 means only twenty simultaneous voice shoppers platform-wide unless quota is raised - everyone else gets text fallback. Fourth is our SSE gateway and Redis pub/sub fan-out - 500K EventSource connections require dedicated edge scaling. Postgres survives if reads dominate; cart writes spike on promotional moments. Mitigation: aggressive CDN latch, raise shard count at schedule time, queue AI admission with transparent UX, scale SSE horizontally.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

Q12. Customer wants Azure OpenAI/Gemini instead of Agora managed LLM

PART 2 - Requirement Change Scenarios

ConvoAI join body in convoai.ts currently sets credential_mode managed for OpenAI-compatible LLM with enable_tools true. Switching to Azure OpenAI or Gemini is a ConvoAI credential configuration change plus validation that tool calling and MCP transport remain supported - the commerce boundary unchanged.

Our MCP server, HMAC auth, and toolExecutor.ts stay identical; only the planner inside Agora changes. If the customer requires direct LLM API ownership outside ConvoAI entirely, we would fall back to the prototype's LLM_PROVIDER text path extended to a custom voice loop - losing managed barge-in and increasing latency. Architecturally I would keep MCP as the execution contract so provider swaps do not rewrite cart logic. Studio pipeline ID may override prompts; document who owns system prompt in production. Test idempotency and get_live_offer accuracy after migration.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

Q13. PSTN access for shoppers without smartphones

PART 2 - Requirement Change Scenarios

PSTN would sit outside the current browser-first prototype as an alternate access channel into the same commerce backend. Agora offers PSTN bridging in some product bundles; architecturally a phone caller would map to a synthetic `userId`, receive DTMF or limited voice menu, and connect to ConvoAI via SIP or a telephony adapter rather than dual `IAgoraRTCCClient` instances.

Critical gaps: no `VideoStage.tsx`, no RTM chat shards, eligibility still via `resolveLineContext` when `add_to_cart` includes `liveSessionId` captured from IVR session binding. Discount enforcement remains server-side. PSTN latency and codec differences affect barge-in tuning. Record consent via spoken prompt. Scale ConvoAI agents separately from PSTN concurrent calls.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: `Media Push` and `deliveryTier.ts` handle scale, RTM sharding in `lifecycle.ts` handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with `requiresLiveSession` remains enforceable because `eligibility.ts` evaluates truth at cart-read time rather than trusting browser or LLM state.

Q14. Stricter privacy GDPR + India DPDP - no voice retention

PART 2 - Requirement Change Scenarios

Under GDPR and India DPDP with zero voice retention, we disable Cloud Recording for AI channels, set RTT to ephemeral caption mode only, and ensure ConvoAI transcripts on RTM are not persisted - process in memory for UI display then discard. MCP logs redact utterances; store only toolCallId, tool name, argsHash, and outcome codes in Postgres for audit without audio or raw text.

Data processing agreements cover Agora as processor; data residency selects EU or India Agora regions. Consent banners separate marketing, analytics, and AI processing. Right to erasure deletes conversation rows and cart linkage; Redis ai:agents leases expire naturally. Eligibility and promotions unchanged - they never depended on voice storage.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and deliveryTier.ts handle scale, RTM sharding in lifecycle.ts handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with requiresLiveSession remains enforceable because eligibility.ts evaluates truth at cart-read time rather than trusting browser or LLM state.

Q15. Discount changes to 15% electronics only, stackable with coupon, max cap

PART 2 - Requirement Change Scenarios

The promotion engine in `packages/shared/src/promotions.ts` already supports category conditions, stackable flags, priority ordering, and `MAX_TOTAL_DISCOUNT_PCT` cap at 50%. Admin PATCH updates LIVE20 to 15%, adds `categorySlug` filter for electronics, sets stackable true, and defines max discount minor units per line or order.

`resolveLineContext` supplies `categorySlug` from the SQL join; `evaluatePromotions` computes best stackable combination without exceeding cap. SSE `promotions.changed` triggers client refetch. AI tools `get_live_offer` returns structured eligible flag from same evaluator - never hallucinated percentages. Session attachment rules unchanged.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and `deliveryTier.ts` handle scale, RTM sharding in `lifecycle.ts` handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with `requiresLiveSession` remains enforceable because `eligibility.ts` evaluates truth at cart-read time rather than trusting browser or LLM state.

Q16. Discount only if checkout within 30 minutes of add-to-cart during live

PART 2 - Requirement Change Scenarios

Introduce a `cart_line_live_window` table or column `liveAddedAt` captured in `add_to_cart` when `liveEligible` is true. `evaluatePromotions` adds `requiresCheckoutWithinMinutes` condition checked against `ctx.now` from `resolveLineContext`.

Session end still reprices carts via `lifecycle.ts`, but a line added at minute 10 during live remains eligible until `liveAddedAt` plus 30 minutes even if session ends at minute 60 - product decision documented. Alternatively strict interpretation ties window to session status live only; document choice. SSE `cart.updated` on expiry job removes discount. AI `get_live_offer` reads computed eligibility, not timers from LLM.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and `deliveryTier.ts` handle scale, RTM sharding in `lifecycle.ts` handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with `requiresLiveSession` remains enforceable because `eligibility.ts` evaluates truth at cart-read time rather than trusting browser or LLM state.

Q17. Why two RTC channels per viewer instead of one?

PART 3 - Agora Product Judgment

ConvoAI join in convoai.ts requires remote_rtc_uids with exactly one shopper UID - the agent publishes TTS audio privately to ai-<conversationId>. The public live host stream on live-<slug> must remain audible to all audience members simultaneously; mixing private AI speech into that channel would leak one shopper's conversation to thousands.

VideoStage.tsx creates IAgoraRTCClient #1 as audience on live-<slug>; useVoiceAgent.ts creates client #2 as publisher on ai-<conversationId> for microphone input. Audio ducking lowers host track volume locally while agent speaks. Tokens mint separately via tokens.ts with channel ownership checks. This is not inefficiency - it is privacy and API constraint compliance.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and deliveryTier.ts handle scale, RTM sharding in lifecycle.ts handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with requiresLiveSession remains enforceable because eligibility.ts evaluates truth at cart-read time rather than trusting browser or LLM state.

Q18. Why RTM/Signaling for chat instead of RTC data streams or WebSocket?

PART 3 - Agora Product Judgment

RTM provides channel semantics matching our sharded chat-<slug>-<index> model, Agora-native scaling, and shared RtmProvider.tsx client for chat plus ConvoAI transcripts without a second websocket infrastructure. RTC data streams would ride on live channel capacity ill-suited for 500K metadata fan-out and would conflate media subscription with chat authorization.

Custom WebSockets would duplicate SSE for server-push and require us to build global presence and moderation sync. Server-side publish via signaling.ts enables host fan-out to all shards and REST fallback degradation to SSE when RTM budget exceeded.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and deliveryTier.ts handle scale, RTM sharding in lifecycle.ts handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with requiresLiveSession remains enforceable because eligibility.ts evaluates truth at cart-read time rather than trusting browser or LLM state.

Q19. Live Streaming vs Interactive Live Streaming vs Broadcast Streaming?

PART 3 - Agora Product Judgment

Interactive Live Streaming (RTC live mode) suits the prototype seller and early audience: sub-second latency, host interaction, co-host potential, and triggers Media Push from the same publisher. Broadcast Streaming optimizes one-to-many with higher latency acceptable for passive viewing - our CDN tier after deliveryTier flip approximates this with HLS. Pure Live Streaming products without RTC interaction would break polls, reactions, and low-latency commerce urgency. We hybridize: RTC until threshold, then CDN latch in useLiveSession.ts cdnLatchedRef.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and deliveryTier.ts handle scale, RTM sharding in lifecycle.ts handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with requiresLiveSession remains enforceable because eligibility.ts evaluates truth at cart-read time rather than trusting browser or LLM state.

Q20. Why not put the AI agent in the same channel as the host?

PART 3 - Agora Product Judgment

Same-channel placement violates private voice commerce: every audience member would hear the shopper's AI responses and potentially their microphone unless complex selective subscription existed - which Agora ConvoAI does not model. `remote_rtc_uids` explicitly targets one UID on a dedicated ai channel. Support escalation in `support.ts` joins the same `ai-<conversationId>` for human handoff without exposing support audio on `live-<slug>`.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and `deliveryTier.ts` handle scale, RTM sharding in `lifecycle.ts` handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with `requiresLiveSession` remains enforceable because `eligibility.ts` evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (`MEDIA_PUSH_ENABLED=false` origin, `RTC_TIER_MAX_VIEWERS=3`, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q21. Cloud Recording vs browser MediaRecorder?

PART 3 - Agora Product Judgment

Browser MediaRecorder in useHostRecorder.ts is the demo-primary path - no extra Agora recording charges, works without RECORDING_PROVIDER=agora, captures what the host browser composes. Cloud Recording in recording.ts provides authoritative mixed stream from Agora servers, better for OBS hosts via Media Gateway, tamper-evident archives, and S3 lifecycle policies - but adds cost, acquire/start latency, and side service failure handling in lifecycle.ts.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and deliveryTier.ts handle scale, RTM sharding in lifecycle.ts handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with requiresLiveSession remains enforceable because eligibility.ts evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (MEDIA_PUSH_ENABLED=false origin, RTC_TIER_MAX_VIEWERS=3, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q22. What does Media Push do that RTC cannot at scale?

PART 3 - Agora Product Judgment

Media Push creates an RTMP converter at go-live pushing host media to MEDIA_PUSH_RTMP_URL/<slug>, originating HLS at MEDIA_PUSH_HLS_URL for mass audience. RTC cannot economically serve 500K interactive subscribers on one channel; CDN distribution amortizes egress. RTC remains for host and early interactive tier; chat, AI, polls stay real-time on parallel channels. Prototype uses MEDIA_PUSH_ENABLED=false with simulated-origin fixtures but real tier flip logic in deliveryTier.ts.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and deliveryTier.ts handle scale, RTM sharding in lifecycle.ts handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with requiresLiveSession remains enforceable because eligibility.ts evaluates truth at cart-read time rather than trusting browser or LLM state.

Q23. Agora Analytics / Content Moderation - would you use them?

PART 3 - Agora Product Judgment

Yes in production. Agora Analytics informs RTC bitrate, freeze rate, and join success correlated with delivery tier transitions. Content Moderation augments our Redis ban/mute sets in chat.ts for image/text toxicity at scale. They complement but do not replace server-side chat authorization and human moderator tools on seller dashboard. Prototype implements basic moderation endpoints; Analytics would be enabled day one in staging.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and deliveryTier.ts handle scale, RTM sharding in lifecycle.ts handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with requiresLiveSession remains enforceable because eligibility.ts evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (MEDIA_PUSH_ENABLED=false origin, RTC_TIER_MAX_VIEWERS=3, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q24. Token renewal for 4-hour live shows?

PART 3 - Agora Product Judgment

tokens.ts mints RTC and RTM tokens with AGORA_TOKEN_TTL_SECONDS; clients must renew before expiry. VideoStage and useVoiceAgent should listen for token-privilege-will-expire and call /api/tokens refresh endpoints in routes/tokens.ts with same channel ownership checks. Four-hour shows require worker cron or client timers; ConvoAI agent token via mintAgentRtcRtmToken renews independently. Failure to renew drops audience silently - monitor join errors in Analytics.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and deliveryTier.ts handle scale, RTM sharding in lifecycle.ts handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with requiresLiveSession remains enforceable because eligibility.ts evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (MEDIA_PUSH_ENABLED=false origin, RTC_TIER_MAX_VIEWERS=3, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q25. Co-host / multi-host scenario?

PART 3 - Agora Product Judgment

Multiple publishers on live-<slug> with publisher role tokens; each host UID allocated from `agora_uid_seq`. Co-host audio mixes in channel; Media Push converter captures composite. Chat host messages still fan out from seller account. Eligibility and promotions unaffected. Token routes must authorize each host `userId` against session `sellerId` or `cohost` list.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and `deliveryTier.ts` handle scale, RTM sharding in `lifecycle.ts` handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with `requiresLiveSession` remains enforceable because `eligibility.ts` evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (`MEDIA_PUSH_ENABLED=false` origin, `RTC_TIER_MAX_VIEWERS=3`, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q26. When to use Media Gateway for OBS/RTMP ingest?

PART 3 - Agora Product Judgment

When sellers require professional cameras, scenes, and overlays via OBS rather than browser getUserMedia in useHostBroadcast.ts. mediagateway.ts provides RTMP ingest into Agora; browser replay path differs - documented in README. Use when stream quality, stability, or branding exceeds browser capture; accept added setup complexity and latency tuning.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and deliveryTier.ts handle scale, RTM sharding in lifecycle.ts handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with requiresLiveSession remains enforceable because eligibility.ts evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (MEDIA_PUSH_ENABLED=false origin, RTC_TIER_MAX_VIEWERS=3, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q27. What is the role of Real-Time STT (RTT) vs ConvoAI ASR?

PART 3 - Agora Product Judgment

RTT in `rtt.ts` transcribes the public live stream on `live-<slug>` for host captions via `pub bot - session.rtt_status_changed` SSE. ConvoAI managed ASR in `join` body transcribes shopper microphone for tool planning on `ai` channel. They serve different speakers and outputs: RTT is display captions; ConvoAI ASR is dialog understanding. Disable RTT if captions not required to save cost.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: `Media Push` and `deliveryTier.ts` handle scale, RTM sharding in `lifecycle.ts` handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and `LIVE20` with `requiresLiveSession` remains enforceable because `eligibility.ts` evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (`MEDIA_PUSH_ENABLED=false` origin, `RTC_TIER_MAX_VIEWERS=3`, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q28. Why hybrid RTC+CDN instead of CDN-only for live commerce?

PART 3 - Agora Product Judgment

CDN-only HLS sacrifices sub-second interaction, delaying urgency mechanics and host rapport during early audience phase when RTC is affordable. Hybrid preserves interactive tier for first N viewers (RTC_TIER_MAX_VIEWERS) then flips once for cost control. Commerce chat, private AI, and cart SSE remain real-time regardless. CDN-only would still work for passive QVC-style but weaken prototype differentiation.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and deliveryTier.ts handle scale, RTM sharding in lifecycle.ts handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with requiresLiveSession remains enforceable because eligibility.ts evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (MEDIA_PUSH_ENABLED=false origin, RTC_TIER_MAX_VIEWERS=3, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q29. Walk through one full voice turn end-to-end

PART 4 - Voice AI Deep Dives

Shopper taps voice in Live.tsx; useVoiceAgent creates conversation via POST /api/ai/conversations, receives MCP-signed headers, joins ai-<conversationId> as publisher, POST start triggers joinConvoAiAgent. Shopper speaks; ConvoAI ASR segments utterance; managed LLM plans tool call search_products with args; Agora POSTs /mcp with X-Convo-Signature; mcp/auth validates; toolExecutor runs search against Postgres; JSON returns; LLM composes speech; TTS publishes to ai channel; shopper hears response while host audio ducked. Transcript arrives RTM data_channel. If add_to_cart, idempotent write and cart.updated SSE fires.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and deliveryTier.ts handle scale, RTM sharding in lifecycle.ts handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with requiresLiveSession remains enforceable because eligibility.ts evaluates truth at cart-read time rather than trusting browser or LLM state.

Q30. What is MCP and why not plain REST webhooks?

PART 4 - Voice AI Deep Dives

Model Context Protocol standardizes tool discovery, schema, and streamable HTTP transport - mcp/server.ts registers fourteen tools with @modelcontextprotocol/sdk. Plain webhooks lack uniform tool listing, complicate idempotency correlation, and push schema drift. MCP gives Agora ConvoAI native enable_tools integration with allowed_tools whitelist.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and deliveryTier.ts handle scale, RTM sharding in lifecycle.ts handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with requiresLiveSession remains enforceable because eligibility.ts evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (MEDIA_PUSH_ENABLED=false origin, RTC_TIER_MAX_VIEWERS=3, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q31. How prevent LLM hallucinating prices?

PART 4 - Voice AI Deep Dives

Never embed catalog prices in prompts as authoritative; systemPrompt.ts states rules but numeric prices only from tool results. `get_product_details` and `add_to_cart` call `resolveLineContext` and `evaluatePromotions`. `get_live_offer` returns structured eligible boolean and computed discount from Postgres. If session ended, `liveEligible` false and tool response says ineligible - LLM must read tool JSON.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: `Media Push` and `deliveryTier.ts` handle scale, RTM sharding in `lifecycle.ts` handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with `requiresLiveSession` remains enforceable because `eligibility.ts` evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (`MEDIA_PUSH_ENABLED=false` origin, `RTC_TIER_MAX_VIEWERS=3`, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q32. Barge-in while agent is speaking?

PART 4 - Voice AI Deep Dives

ConvoAI managed pipeline handles barge-in natively - shopper speech interrupts TTS. Client enables microphone on ai channel continuously during active conversation. Audio ducking on host track is independent. Test with MiniMax TTS configured in convoai.ts; tune sensitivity in Studio if AGORA_STUDIO_PIPELINE_ID set.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and deliveryTier.ts handle scale, RTM sharding in lifecycle.ts handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with requiresLiveSession remains enforceable because eligibility.ts evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (MEDIA_PUSH_ENABLED=false origin, RTC_TIER_MAX_VIEWERS=3, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q33. ConvoAI quota exhaustion handling?

PART 4 - Voice AI Deep Dives

admission.ts Redis sorted set ai:agents enforces CONVOAI_MAX_CONCURRENT_AGENTS; excess receives 503 ai_capacity. UI falls back to LLM_PROVIDER text chat on Commerce API reusing same tools. Worker background.ts sweeps expired leases. Monitor ai:agents ZCARD; scale quota with Agora billing and horizontal AI replicas with nginx sticky MCP.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and deliveryTier.ts handle scale, RTM sharding in lifecycle.ts handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with requiresLiveSession remains enforceable because eligibility.ts evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (MEDIA_PUSH_ENABLED=false origin, RTC_TIER_MAX_VIEWERS=3, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q34. Studio vs programmatic join - who owns system prompt in production?

PART 4 - Voice AI Deep Dives

Development: programmatic systemPrompt.ts and MCP headers for reproducibility. Production: split - Studio owns persona/tone/pipeline when AGORA_STUDIO_PIPELINE_ID set; engineering owns MCP tools, HMAC secrets, allowed_tools, and commerce truth. Change control: Studio updates require regression on get_live_offer accuracy.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and deliveryTier.ts handle scale, RTM sharding in lifecycle.ts handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with requiresLiveSession remains enforceable because eligibility.ts evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (MEDIA_PUSH_ENABLED=false origin, RTC_TIER_MAX_VIEWERS=3, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q35. Why voice-first but still have text mode?

PART 4 - Voice AI Deep Dives

Voice suits live shopping hands-busy context; text covers ConvoAI capacity exhaustion, noisy environments, accessibility, and users who prefer reading transcripts. Same toolExecutor backend ensures parity.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and deliveryTier.ts handle scale, RTM sharding in lifecycle.ts handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with requiresLiveSession remains enforceable because eligibility.ts evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (MEDIA_PUSH_ENABLED=false origin, RTC_TIER_MAX_VIEWERS=3, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q36. Multi-language voice assistance architecture?

PART 4 - Voice AI Deep Dives

ConvoAI ASR/TTS language params in join body; tool responses remain locale-agnostic JSON; UI i18n separate. RTT language per session config. Eligibility logic language-independent.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and deliveryTier.ts handle scale, RTM sharding in lifecycle.ts handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with requiresLiveSession remains enforceable because eligibility.ts evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (MEDIA_PUSH_ENABLED=false origin, RTC_TIER_MAX_VIEWERS=3, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q37. What happens when PUBLIC_API_URL/mcp is unreachable?

PART 4 - Voice AI Deep Dives

ConvoAI tool calls fail; agent apologizes per LLM behavior; logs MCP timeouts. Shopper can still watch live and use text fallback if API up. Monitor MCP 5xx; ngrok misconfig common in demo - see Q105.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and deliveryTier.ts handle scale, RTM sharding in lifecycle.ts handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with requiresLiveSession remains enforceable because eligibility.ts evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (MEDIA_PUSH_ENABLED=false origin, RTC_TIER_MAX_VIEWERS=3, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q38. Tool call idempotency - why and how?

PART 4 - Voice AI Deep Dives

Network retries duplicate `add_to_cart` without idempotency. `toolExecutor.ts` claims (`conversationId`, `toolCallId`) with `argsHash`; replay returns prior result. Essential for cart correctness and payment-grade mutations.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and `deliveryTier.ts` handle scale, RTM sharding in `lifecycle.ts` handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with `requiresLiveSession` remains enforceable because `eligibility.ts` evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (`MEDIA_PUSH_ENABLED=false` origin, `RTC_TIER_MAX_VIEWERS=3`, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q39. User adds at minute 59, checkout at minute 61 after session ends - what price?

PART 5 - Business Logic & Commerce

Cart line stores `liveSessionId` from add time. At checkout minute 61, `resolveLineContext` sees `session_status` ended, `attached` true but `liveEligible` false. `LIVE20 requiresLiveSession` fails in `evaluatePromotions`. Shopper pays undiscounted selling price unless separate window rule exists.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: `Media Push` and `deliveryTier.ts` handle scale, RTM sharding in `lifecycle.ts` handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and `LIVE20` with `requiresLiveSession` remains enforceable because `eligibility.ts` evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (`MEDIA_PUSH_ENABLED=false` origin, `RTC_TIER_MAX_VIEWERS=3`, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q40. Two different live sessions in one cart - which discount?

PART 5 - Business Logic & Commerce

Each line carries its own `liveSessionId`; `resolveLineContext` evaluates per line. Line from live A eligible only if A live and product attached to A; line from live B independent. Order-level promotions consider subtotal aggregation rules in `evaluatePromotions`.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and `deliveryTier.ts` handle scale, RTM sharding in `lifecycle.ts` handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with `requiresLiveSession` remains enforceable because `eligibility.ts` evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (`MEDIA_PUSH_ENABLED=false` origin, `RTC_TIER_MAX_VIEWERS=3`, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q41. Product featured in live A but user joined live B - eligible?

PART 5 - Business Logic & Commerce

No. `liveEligible` requires product attached to the specific `liveSessionId` on the cart line, not merely featured elsewhere. Joining live B does not attach products from live A.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and `deliveryTier.ts` handle scale, RTM sharding in `lifecycle.ts` handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with `requiresLiveSession` remains enforceable because `eligibility.ts` evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (`MEDIA_PUSH_ENABLED=false` origin, `RTC_TIER_MAX_VIEWERS=3`, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q42. AI says 20% off but session ended 2 seconds ago - prevention?

PART 5 - Business Logic & Commerce

Tool `get_live_offer` and `add_to_cart` re-query `resolveLineContext` at execution instant; LLM narration cannot commit discount. Session end triggers cart repricing via `lifecycle.ts`. Instruct prompt: always confirm via tool before stating price.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and `deliveryTier.ts` handle scale, RTM sharding in `lifecycle.ts` handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with `requiresLiveSession` remains enforceable because `eligibility.ts` evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (`MEDIA_PUSH_ENABLED=false` origin, `RTC_TIER_MAX_VIEWERS=3`, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q43. Can user refresh page to keep discount after session ends?

PART 5 - Business Logic & Commerce

No durable cheat: cart in Postgres repriced on session end; refresh refetches server cart without LIVE20. `liveSessionId` on line still references ended session where `liveEligible` false.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and `deliveryTier.ts` handle scale, RTM sharding in `lifecycle.ts` handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with `requiresLiveSession` remains enforceable because `eligibility.ts` evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (`MEDIA_PUSH_ENABLED=false` origin, `RTC_TIER_MAX_VIEWERS=3`, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q44. Discount on MRP or selling price?

PART 5 - Business Logic & Commerce

`evaluatePromotions` applies percent to `unitPriceMinorUnits` from variant row (selling price), not `mrp_minor_units` unless business rule adds MRP-based promos. `resolveLineContext` exposes both for display.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: `Media Push` and `deliveryTier.ts` handle scale, RTM sharding in `lifecycle.ts` handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with `requiresLiveSession` remains enforceable because `eligibility.ts` evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (`MEDIA_PUSH_ENABLED=false` origin, `RTC_TIER_MAX_VIEWERS=3`, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q45. Stock runs out while AI is talking?

PART 5 - Business Logic & Commerce

`add_to_cart` checks `variantStock` in `resolveLineContext`; insufficient stock returns tool error JSON; LLM informs shopper. No oversell without negative stock rule.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and `deliveryTier.ts` handle scale, RTM sharding in `lifecycle.ts` handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with `requiresLiveSession` remains enforceable because `eligibility.ts` evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (`MEDIA_PUSH_ENABLED=false` origin, `RTC_TIER_MAX_VIEWERS=3`, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q46. User asks to add 10 units but only 2 in stock?

PART 5 - Business Logic & Commerce

Tool validates quantity against variantStock; reject or partial add per product policy encoded in tool handler - prototype rejects over quantity with structured error.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and deliveryTier.ts handle scale, RTM sharding in lifecycle.ts handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with requiresLiveSession remains enforceable because eligibility.ts evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (MEDIA_PUSH_ENABLED=false origin, RTC_TIER_MAX_VIEWERS=3, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q47. Guest checkout vs logged-in for voice AI?

PART 5 - Business Logic & Commerce

Prototype uses session cookie `userId` for both; guest would map to anonymous user with same MCP auth bound to conversation. Logged-in enables order history tools `list_my_orders`.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and `deliveryTier.ts` handle scale, RTM sharding in `lifecycle.ts` handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with `requiresLiveSession` remains enforceable because `eligibility.ts` evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (`MEDIA_PUSH_ENABLED=false` origin, `RTC_TIER_MAX_VIEWERS=3`, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q48. Real PCI boundary for payments?

PART 5 - Business Logic & Commerce

Payments mocked in prototype; production delegates to PCI DSS compliant PSP hosted fields or redirect - card data never touches Commerce API or MCP. `get_payment_options` returns tokenization metadata only.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and `deliveryTier.ts` handle scale, RTM sharding in `lifecycle.ts` handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with `requiresLiveSession` remains enforceable because `eligibility.ts` evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (`MEDIA_PUSH_ENABLED=false` origin, `RTC_TIER_MAX_VIEWERS=3`, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q49. Idempotency - user says add to cart twice quickly?

PART 5 - Business Logic & Commerce

Duplicate utterances may produce separate toolCallIds - two lines unless same idempotency key. ConvoAI may reuse toolCallId on retry; argsHash dedupes identical mutations. Product may merge quantities in cart.ts.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and deliveryTier.ts handle scale, RTM sharding in lifecycle.ts handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with requiresLiveSession remains enforceable because eligibility.ts evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (MEDIA_PUSH_ENABLED=false origin, RTC_TIER_MAX_VIEWERS=3, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q50. Should AI remove from cart or apply coupon codes?

PART 5 - Business Logic & Commerce

Scope minimization: prototype tools `add_to_cart`, `get_cart`, `not remove` or `apply_coupon` - reduces risk. Production adds `remove_line` and `apply_coupon` as separate audited tools with confirmation for high impact.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: `Media Push` and `deliveryTier.ts` handle scale, RTM sharding in `lifecycle.ts` handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with `requiresLiveSession` remains enforceable because `eligibility.ts` evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (`MEDIA_PUSH_ENABLED=false` origin, `RTC_TIER_MAX_VIEWERS=3`, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q51. High-ticket items - require explicit confirmation?

PART 5 - Business Logic & Commerce

Yes for production: threshold in `add_to_cart` returns `pending_confirmation` status; second `confirm_purchase` tool or verbal yes with `confirm` toolCall. Prototype may skip but architect recommends for jewelry/electronics.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: `Media Push` and `deliveryTier.ts` handle scale, RTM sharding in `lifecycle.ts` handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with `requiresLiveSession` remains enforceable because `eligibility.ts` evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (`MEDIA_PUSH_ENABLED=false` origin, `RTC_TIER_MAX_VIEWERS=3`, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q52. Malicious MCP calls outside Agora?

PART 5 - Business Logic & Commerce

HMAC X-Convo-Signature without secret fails mcp/auth.ts. Static bearer scoped to conversation. Rate limit by conversationId and IP. No admin tools in MCP_ALLOWED_TOOLS.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and deliveryTier.ts handle scale, RTM sharding in lifecycle.ts handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with requiresLiveSession remains enforceable because eligibility.ts evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (MEDIA_PUSH_ENABLED=false origin, RTC_TIER_MAX_VIEWERS=3, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q53. Walk through stages 1-11 - what's missing in prototype?

PART 6 - Customer Journey & Product

Stages through browse, join live, chat, voice AI, cart, checkout largely exist. Gaps: real payments, full seller analytics dashboard, advanced recommendations ML, mobile native apps, PSTN, full Content Moderation integration, and end-to-end Agora Media Push with real CDN origin.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and deliveryTier.ts handle scale, RTM sharding in lifecycle.ts handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with requiresLiveSession remains enforceable because eligibility.ts evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (MEDIA_PUSH_ENABLED=false origin, RTC_TIER_MAX_VIEWERS=3, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q54. Why voice AI during live instead of public chat?

PART 6 - Customer Journey & Product

Public chat shards do not scale private product questions; voice AI gives each shopper personalized catalog access via MCP without spamming channel. Host keeps broadcast narrative; AI handles sizing, stock, delivery privately.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and `deliveryTier.ts` handle scale, RTM sharding in `lifecycle.ts` handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with `requiresLiveSession` remains enforceable because `eligibility.ts` evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (`MEDIA_PUSH_ENABLED=false` origin, `RTC_TIER_MAX_VIEWERS=3`, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q55. Replay + voice AI value without live discount?

PART 6 - Customer Journey & Product

Recorded sessions enable asynchronous shopping; voice AI without LIVE20 still assists product discovery from replay context linking recorded session products. Eligibility requires live requiresLiveSession false for replay-specific promos.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and deliveryTier.ts handle scale, RTM sharding in lifecycle.ts handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with requiresLiveSession remains enforceable because eligibility.ts evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (MEDIA_PUSH_ENABLED=false origin, RTC_TIER_MAX_VIEWERS=3, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q56. Browse-only voice AI vs ChatGPT with plugin?

PART 6 - Customer Journey & Product

In-platform AI inherits `liveSessionId`, `cart`, `LIVE20` eligibility, and `escalate_to_human` to support queue on same ai channel - ChatGPT plugin lacks integrated commerce state and Agora latency profile.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and `deliveryTier.ts` handle scale, RTM sharding in `lifecycle.ts` handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and `LIVE20` with `requiresLiveSession` remains enforceable because `eligibility.ts` evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (`MEDIA_PUSH_ENABLED=false` origin, `RTC_TIER_MAX_VIEWERS=3`, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q57. How does host benefit from viewers using private AI?

PART 6 - Customer Journey & Product

Higher conversion without chat noise; AI drives add_to_cart events visible in seller metrics; host focuses presentation while AI closes product questions. Aggregated intent signals possible without exposing private transcripts.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and deliveryTier.ts handle scale, RTM sharding in lifecycle.ts handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with requiresLiveSession remains enforceable because eligibility.ts evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (MEDIA_PUSH_ENABLED=false origin, RTC_TIER_MAX_VIEWERS=3, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q58. Engagement mechanisms beyond chat - why polls/reactions/pinned products?

PART 6 - Customer Journey & Product

Worker aggregates reactions at 1Hz; polls drive participation without RTM flood; pinned products align featured SKU with eligibility attachment in `live_session_products` - reducing AI/catalog mismatch.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and `deliveryTier.ts` handle scale, RTM sharding in `lifecycle.ts` handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with `requiresLiveSession` remains enforceable because `eligibility.ts` evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (`MEDIA_PUSH_ENABLED=false` origin, `RTC_TIER_MAX_VIEWERS=3`, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q59. Seller scheduling and attaching products before go-live?

PART 6 - Customer Journey & Product

Sessions scheduled in lifecycle; products attached to live_session_products before start ensures resolveLineContext attached flag true at go-live. Seller UI selects catalog subset defining AI offer scope.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and deliveryTier.ts handle scale, RTM sharding in lifecycle.ts handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with requiresLiveSession remains enforceable because eligibility.ts evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (MEDIA_PUSH_ENABLED=false origin, RTC_TIER_MAX_VIEWERS=3, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q60. What seller sees during live - analytics and moderation?

PART 6 - Customer Journey & Product

Seller view: viewer count from Redis flush SSE, chat moderation bans/mutes, poll results, delivery tier indicator. Production adds Agora Analytics dashboard and recording status from recordingStatus column.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and deliveryTier.ts handle scale, RTM sharding in lifecycle.ts handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with requiresLiveSession remains enforceable because eligibility.ts evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (MEDIA_PUSH_ENABLED=false origin, RTC_TIER_MAX_VIEWERS=3, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q61. Session lifecycle start/end - idempotency and side effects?

PART 7 - Implementation Walkthrough

lifecycle.ts conditional UPDATE scheduled to live with pg_advisory_xact_lock; side effects startRecording, startRtt, createConverter idempotent with status columns; end reprices carts, stops services, queues summary job; reconcileSessionEffects heals partial failures.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and deliveryTier.ts handle scale, RTM sharding in lifecycle.ts handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with requiresLiveSession remains enforceable because eligibility.ts evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (MEDIA_PUSH_ENABLED=false origin, RTC_TIER_MAX_VIEWERS=3, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q62. Where are RTC/RTM tokens minted and what roles?

PART 7 - Implementation Walkthrough

`packages/agora/src/tokens.ts` `mintRtcToken` audience level 2 for viewers, publisher for host, `mintRtmToken` `user-<userId>`, `mintAgentRtcRtmToken` for ConvoAI agent. `apps/api/src/routes/tokens.ts` enforces channel ownership before mint.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and `deliveryTier.ts` handle scale, RTM sharding in `lifecycle.ts` handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with `requiresLiveSession` remains enforceable because `eligibility.ts` evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (`MEDIA_PUSH_ENABLED=false` origin, `RTC_TIER_MAX_VIEWERS=3`, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q63. ConvoAI join payload - MCP headers, channel, remote_rtc_uids?

PART 7 - Implementation Walkthrough

convoai.ts: channel ai-<conversationId>, remote_rtc_uids [viewerUid], MCP streamable_http endpoint PUBLIC_API_URL/mcp, buildMcpAuthHeaders X-Convo-Signature, allowed_tools, enable_rtm data_channel rtm, managed ASR LLM TTS.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and deliveryTier.ts handle scale, RTM sharding in lifecycle.ts handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with requiresLiveSession remains enforceable because eligibility.ts evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (MEDIA_PUSH_ENABLED=false origin, RTC_TIER_MAX_VIEWERS=3, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q64. MCP auth - X-Convo-Signature verification?

PART 7 - Implementation Walkthrough

callbackAuth.ts HMAC-SHA256 conversationId.expires base64url; mcp/auth.ts validates expiry and signature; optional MCP_STATIC_API_KEY bearer for Console tests; nginx sticky X-Convo-Id.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and deliveryTier.ts handle scale, RTM sharding in lifecycle.ts handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with requiresLiveSession remains enforceable because eligibility.ts evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (MEDIA_PUSH_ENABLED=false origin, RTC_TIER_MAX_VIEWERS=3, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q65. Promotion engine requiresLiveSession evaluation?

PART 7 - Implementation Walkthrough

packages/shared/src/promotions.ts: if requiresLiveSession && !ctx.liveEligible return false. ctx from resolveLineContext liveEligible SQL proof. LIVE20 seeded requiresLiveSession true in seed.ts.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and deliveryTier.ts handle scale, RTM sharding in lifecycle.ts handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with requiresLiveSession remains enforceable because eligibility.ts evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (MEDIA_PUSH_ENABLED=false origin, RTC_TIER_MAX_VIEWERS=3, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q66. Eligibility SQL proving product attached to session?

PART 7 - Implementation Walkthrough

eligibility.ts loadLine left join live_sessions on line liveSessionId left join live_session_products on session and product id; attached boolean; liveEligible requires status live and attached true.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and deliveryTier.ts handle scale, RTM sharding in lifecycle.ts handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with requiresLiveSession remains enforceable because eligibility.ts evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (MEDIA_PUSH_ENABLED=false origin, RTC_TIER_MAX_VIEWERS=3, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q67. SSE vs WebSocket vs polling - why SSE?

PART 7 - Implementation Walkthrough

packages/platform/src/lib/sse.ts publishes Redis; sse-gateway streams EventSource; one direction fan-out scales; clients refetch snapshots on reconnect; avoids WebSocket sticky state on API servers.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and deliveryTier.ts handle scale, RTM sharding in lifecycle.ts handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with requiresLiveSession remains enforceable because eligibility.ts evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (MEDIA_PUSH_ENABLED=false origin, RTC_TIER_MAX_VIEWERS=3, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q68. Two IAgoraRTCClient instances in React?

PART 7 - Implementation Walkthrough

VideoStage.tsx live client audience on live-<slug>; useVoiceAgent.ts ai client publisher on ai-<conversationId>; separate lifecycles prevent channel mode conflicts; RtmProvider separate RTM client.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and deliveryTier.ts handle scale, RTM sharding in lifecycle.ts handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with requiresLiveSession remains enforceable because eligibility.ts evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (MEDIA_PUSH_ENABLED=false origin, RTC_TIER_MAX_VIEWERS=3, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q69. Audio ducking when AI speaks?

PART 7 - Implementation Walkthrough

Live.tsx passes host remote audio track as duckTrack; useVoiceAgent sets volume 15 DUCKED_VOLUME while agent speaking, 100 FULL_VOLUME after; local playback only.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and deliveryTier.ts handle scale, RTM sharding in lifecycle.ts handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with requiresLiveSession remains enforceable because eligibility.ts evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (MEDIA_PUSH_ENABLED=false origin, RTC_TIER_MAX_VIEWERS=3, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q70. Recording - what's real vs mocked?

PART 7 - Implementation Walkthrough

Browser MediaRecorder real capture; Cloud Recording real when RECORDING_PROVIDER=agora; payments mocked; Media Push origin simulated when MEDIA_PUSH_ENABLED false; promotion engine and MCP tools real.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and deliveryTier.ts handle scale, RTM sharding in lifecycle.ts handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with requiresLiveSession remains enforceable because eligibility.ts evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (MEDIA_PUSH_ENABLED=false origin, RTC_TIER_MAX_VIEWERS=3, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q71. What to delete before production?

PART 7 - Implementation Walkthrough

Remove MCP_STATIC_API_KEY defaults, simulated HLS fixtures, demo RTC_TIER_MAX_VIEWERS=3, seed admin passwords, ngrok docs from prod env, disable verbose debug logs, enforce TLS and secret rotation.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and deliveryTier.ts handle scale, RTM sharding in lifecycle.ts handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with requiresLiveSession remains enforceable because eligibility.ts evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (MEDIA_PUSH_ENABLED=false origin, RTC_TIER_MAX_VIEWERS=3, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q72. Without Agora credentials what still works?

PART 7 - Implementation Walkthrough

Commerce API cart catalog promotions, SSE, Postgres flows; Agora join fails; text LLM_PROVIDER may work if configured; unit checks in `__checks__` skip live tests.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and `deliveryTier.ts` handle scale, RTM sharding in `lifecycle.ts` handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with `requiresLiveSession` remains enforceable because `eligibility.ts` evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (`MEDIA_PUSH_ENABLED=false` origin, `RTC_TIER_MAX_VIEWERS=3`, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q73. Live captions / host transcription with RTT?

PART 8 - Optional Extensions

Enable RTT via `rtt.ts` pub bot on `live-<slug>`; `session.rtt_status_changed` SSE; captions UI subscribes worker aggregates; independent from ConvoAI shopper ASR.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and `deliveryTier.ts` handle scale, RTM sharding in `lifecycle.ts` handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with `requiresLiveSession` remains enforceable because `eligibility.ts` evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (`MEDIA_PUSH_ENABLED=false` origin, `RTC_TIER_MAX_VIEWERS=3`, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q74. Searchable transcript for recorded events?

PART 8 - Optional Extensions

Post-process Cloud Recording with batch STT indexing to Elasticsearch; link timestamps to `live_session_products` for shoppable replay markers.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and `deliveryTier.ts` handle scale, RTM sharding in `lifecycle.ts` handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with `requiresLiveSession` remains enforceable because `eligibility.ts` evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (`MEDIA_PUSH_ENABLED=false` origin, `RTC_TIER_MAX_VIEWERS=3`, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q75. Synchronized product cards during stream?

PART 8 - Optional Extensions

Seller pins product SSE `session.product_pinned`; client highlights card; attachment table ensures eligibility; AI `get_conversation_context` includes pinned ids.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and `deliveryTier.ts` handle scale, RTM sharding in `lifecycle.ts` handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with `requiresLiveSession` remains enforceable because `eligibility.ts` evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (`MEDIA_PUSH_ENABLED=false` origin, `RTC_TIER_MAX_VIEWERS=3`, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q76. Multi-language voice with captions?

PART 8 - Optional Extensions

RTT language per session; ConvoAI TTS voice selection per locale; tool JSON locale neutral; subtitle track from RTT transcript stream.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and `deliveryTier.ts` handle scale, RTM sharding in `lifecycle.ts` handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with `requiresLiveSession` remains enforceable because `eligibility.ts` evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (`MEDIA_PUSH_ENABLED=false` origin, `RTC_TIER_MAX_VIEWERS=3`, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q77. Authentication and seller role enforcement?

PART 8 - Optional Extensions

Session cookie host-scoped; seller routes verify sellerId on session; token mint checks user owns or cohosts session; production adds OAuth and RBAC.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and deliveryTier.ts handle scale, RTM sharding in lifecycle.ts handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with requiresLiveSession remains enforceable because eligibility.ts evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (MEDIA_PUSH_ENABLED=false origin, RTC_TIER_MAX_VIEWERS=3, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q78. Moderation - banned user can still watch or use AI?

PART 8 - Optional Extensions

chat.ts Redis session bans block postChatMessage; watch RTC may continue; AI MCP should check ban set in tool handlers for add_to_cart escalation policy.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and deliveryTier.ts handle scale, RTM sharding in lifecycle.ts handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with requiresLiveSession remains enforceable because eligibility.ts evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (MEDIA_PUSH_ENABLED=false origin, RTC_TIER_MAX_VIEWERS=3, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q79. Wishlist and recommendations in AI?

PART 8 - Optional Extensions

Tools `add_to_wishlist` and `recommend_products` registered in `mcp/server.ts`; segments has `_wishlist` in eligibility for WISHLIST5 promo.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and `deliveryTier.ts` handle scale, RTM sharding in `lifecycle.ts` handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with `requiresLiveSession` remains enforceable because `eligibility.ts` evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (`MEDIA_PUSH_ENABLED=false` origin, `RTC_TIER_MAX_VIEWERS=3`, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q80. Personalized offers without being creepy?

PART 8 - Optional Extensions

`get_personalized_offers` uses segment flags only - `first_order`, `loyalty` - not invasive scraping; disclose in prompt; opt-out flag in user profile.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: `Media Push` and `deliveryTier.ts` handle scale, RTM sharding in `lifecycle.ts` handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with `requiresLiveSession` remains enforceable because `eligibility.ts` evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (`MEDIA_PUSH_ENABLED=false` origin, `RTC_TIER_MAX_VIEWERS=3`, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q81. Where is App Certificate stored?

PART 9 - Production Security & Reliability

Environment variable on API and worker only - never client or git. tokens.ts reads AGORA_APP_CERTIFICATE server-side for RtcTokenBuilder. Rotate via secret manager.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and deliveryTier.ts handle scale, RTM sharding in lifecycle.ts handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with requiresLiveSession remains enforceable because eligibility.ts evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (MEDIA_PUSH_ENABLED=false origin, RTC_TIER_MAX_VIEWERS=3, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q82. Can viewer subscribe as publisher and hijack stream?

PART 9 - Production Security & Reliability

No if tokens mint audience role level 2 only via routes/tokens.ts ownership checks; publishing requires publisher token tied to seller host UID.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and deliveryTier.ts handle scale, RTM sharding in lifecycle.ts handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with requiresLiveSession remains enforceable because eligibility.ts evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (MEDIA_PUSH_ENABLED=false origin, RTC_TIER_MAX_VIEWERS=3, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q83. Chat abuse - automated vs human moderation?

PART 9 - Production Security & Reliability

Automated ban/mute Redis sets and keyword filters; human seller dashboard for edge cases; production Agora Content Moderation API augment.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and `deliveryTier.ts` handle scale, RTM sharding in `lifecycle.ts` handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with `requiresLiveSession` remains enforceable because `eligibility.ts` evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (`MEDIA_PUSH_ENABLED=false` origin, `RTC_TIER_MAX_VIEWERS=3`, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q84. SSRF risks on MCP endpoint?

PART 9 - Production Security & Reliability

MCP accepts only ConvoAI tool POSTs with HMAC - not open URL fetch. No user-supplied URLs in tool handlers without allowlist.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and `deliveryTier.ts` handle scale, RTM sharding in `lifecycle.ts` handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with `requiresLiveSession` remains enforceable because `eligibility.ts` evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (`MEDIA_PUSH_ENABLED=false` origin, `RTC_TIER_MAX_VIEWERS=3`, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q85. Rate limiting on tool calls and cart APIs?

PART 9 - Production Security & Reliability

Apply per conversationId and userId token bucket on /mcp and /api/cart; prevent LLM loop amplification and cart spam.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and deliveryTier.ts handle scale, RTM sharding in lifecycle.ts handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with requiresLiveSession remains enforceable because eligibility.ts evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (MEDIA_PUSH_ENABLED=false origin, RTC_TIER_MAX_VIEWERS=3, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q86. Agora region down - failover?

PART 9 - Production Security & Reliability

Multi-region Agora projects; DNS failover; read-only commerce degrade; voice and video reconnect UX messaging; session state in Postgres survives.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and `deliveryTier.ts` handle scale, RTM sharding in `lifecycle.ts` handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with `requiresLiveSession` remains enforceable because `eligibility.ts` evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (`MEDIA_PUSH_ENABLED=false` origin, `RTC_TIER_MAX_VIEWERS=3`, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q87. MCP unreachable mid-conversation UX?

PART 9 - Production Security & Reliability

Agent verbal error; text fallback; preserve cart state; retry with exponential backoff on Agora side; alert on MCP error rate.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and `deliveryTier.ts` handle scale, RTM sharding in `lifecycle.ts` handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with `requiresLiveSession` remains enforceable because `eligibility.ts` evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (`MEDIA_PUSH_ENABLED=false` origin, `RTC_TIER_MAX_VIEWERS=3`, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q88. Split brain - two hosts start same session?

PART 9 - Production Security & Reliability

lifecycle conditional UPDATE only one wins; pg_advisory_xact_lock; second receives conflict error; reconcileSessionEffects cleanup.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and deliveryTier.ts handle scale, RTM sharding in lifecycle.ts handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with requiresLiveSession remains enforceable because eligibility.ts evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (MEDIA_PUSH_ENABLED=false origin, RTC_TIER_MAX_VIEWERS=3, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q89. Database down during live - video continues?

PART 9 - Production Security & Reliability

RTC and RTM continue; cart and MCP fail closed; display maintenance banner; queue writes when DB returns.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and `deliveryTier.ts` handle scale, RTM sharding in `lifecycle.ts` handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with `requiresLiveSession` remains enforceable because `eligibility.ts` evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (`MEDIA_PUSH_ENABLED=false` origin, `RTC_TIER_MAX_VIEWERS=3`, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q90. Cost model RTC vs CDN egress vs ConvoAI minutes?

PART 9 - Production Security & Reliability

RTC per participant minute dominates early tier; CDN egress cheaper at scale; ConvoAI billed per agent minute; model break-even at RTC_TIER_MAX_VIEWERS.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and deliveryTier.ts handle scale, RTM sharding in lifecycle.ts handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with requiresLiveSession remains enforceable because eligibility.ts evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (MEDIA_PUSH_ENABLED=false origin, RTC_TIER_MAX_VIEWERS=3, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q91. 100K viewers RTC vs CDN cost difference?

PART 9 - Production Security & Reliability

Orders of magnitude: 100K RTC audience minutes vs single-origin HLS egress plus Media Push converter cost - hybrid flip mandatory.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and `deliveryTier.ts` handle scale, RTM sharding in `lifecycle.ts` handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with `requiresLiveSession` remains enforceable because `eligibility.ts` evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (`MEDIA_PUSH_ENABLED=false` origin, `RTC_TIER_MAX_VIEWERS=3`, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q92. Cap AI cost per session?

PART 9 - Production Security & Reliability

Per-session tool call budget in admission; max conversation duration; degrade to text; monitor ConvoAI minutes per liveSessionId.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and deliveryTier.ts handle scale, RTM sharding in lifecycle.ts handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with requiresLiveSession remains enforceable because eligibility.ts evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (MEDIA_PUSH_ENABLED=false origin, RTC_TIER_MAX_VIEWERS=3, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q93. Is shopper voice stored and for how long?

PART 10 - Privacy & Compliance

Prototype: ConvoAI processes voice ephemerally; RTM transcripts for UI not long-term stored by default. Production policy defines retention; GDPR zero retention disables persistence.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: `Media Push` and `deliveryTier.ts` handle scale, RTM sharding in `lifecycle.ts` handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with `requiresLiveSession` remains enforceable because `eligibility.ts` evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (`MEDIA_PUSH_ENABLED=false` origin, `RTC_TIER_MAX_VIEWERS=3`, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q94. Can support replay private AI conversation?

PART 10 - Privacy & Compliance

Not in prototype without explicit recording consent on ai channel; `escalate_to_human` joins live RTC for real-time assist without automatic replay archive.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and `deliveryTier.ts` handle scale, RTM sharding in `lifecycle.ts` handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with `requiresLiveSession` remains enforceable because `eligibility.ts` evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (`MEDIA_PUSH_ENABLED=false` origin, `RTC_TIER_MAX_VIEWERS=3`, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q95. Children's data considerations?

PART 10 - Privacy & Compliance

Voice AI and profiling require parental consent under COPPA-like regimes; disable personalized offers for minors; age gate at signup.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and `deliveryTier.ts` handle scale, RTM sharding in `lifecycle.ts` handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with `requiresLiveSession` remains enforceable because `eligibility.ts` evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (`MEDIA_PUSH_ENABLED=false` origin, `RTC_TIER_MAX_VIEWERS=3`, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q96. Recording consent banner sufficient for EU?

PART 10 - Privacy & Compliance

Banner plus granular purpose, withdraw link, DPA with Agora, DPIA for voice AI, and separate consent for marketing - not banner alone.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and `deliveryTier.ts` handle scale, RTM sharding in `lifecycle.ts` handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with `requiresLiveSession` remains enforceable because `eligibility.ts` evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (`MEDIA_PUSH_ENABLED=false` origin, `RTC_TIER_MAX_VIEWERS=3`, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q97. Biggest shortcut in prototype?

PART 11 - Trade-offs & Adaptation

RTC_TIER_MAX_VIEWERS=3 and MEDIA_PUSH_ENABLED=false simulated CDN - proves logic not planet scale. Payments mocked. Accept for assignment demo with documented honesty.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and deliveryTier.ts handle scale, RTM sharding in lifecycle.ts handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with requiresLiveSession remains enforceable because eligibility.ts evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (MEDIA_PUSH_ENABLED=false origin, RTC_TIER_MAX_VIEWERS=3, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q98. If 2 more weeks - build first?

PART 11 - Trade-offs & Adaptation

Real Media Push CDN path, production payment tokenization, Agora Analytics dashboards, and load test chat sharding at 10K simulated viewers.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and `deliveryTier.ts` handle scale, RTM sharding in `lifecycle.ts` handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with `requiresLiveSession` remains enforceable because `eligibility.ts` evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (`MEDIA_PUSH_ENABLED=false` origin, `RTC_TIER_MAX_VIEWERS=3`, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q99. Mobile apps next - what breaks?

PART 11 - Trade-offs & Adaptation

Dual RTC client battery and background audio policies; iOS AVAudioSession ducking; push notifications for cart repricing; separate RTM SDK lifecycle.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and `deliveryTier.ts` handle scale, RTM sharding in `lifecycle.ts` handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with `requiresLiveSession` remains enforceable because `eligibility.ts` evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (`MEDIA_PUSH_ENABLED=false` origin, `RTC_TIER_MAX_VIEWERS=3`, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q100. Build vs buy for search, recommendations, payments?

PART 11 - Trade-offs & Adaptation

Buy PSP and search SaaS; build eligibility and live commerce orchestration - that is core IP. Recommendations hybrid: buy embeddings, own live context rules.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and `deliveryTier.ts` handle scale, RTM sharding in `lifecycle.ts` handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with `requiresLiveSession` remains enforceable because `eligibility.ts` evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (`MEDIA_PUSH_ENABLED=false` origin, `RTC_TIER_MAX_VIEWERS=3`, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q101. One thing you would NOT use Agora for?

PART 11 - Trade-offs & Adaptation

Product catalog search indexing, promotion rule authoring UI, or PCI payment vault - keep in customer data plane. Agora for realtime media and voice AI transport only.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through `packages/domain-commerce` and `resolveLineContext` in `eligibility.ts`, which joins `product_variants`, `live_sessions`, and `live_session_products` in one SQL proof at evaluation time. State changes propagate via Redis channels `events:user`, `events:session`, and `events:global` to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and `deliveryTier.ts` handle scale, RTM sharding in `lifecycle.ts` handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with `requiresLiveSession` remains enforceable because `eligibility.ts` evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (`MEDIA_PUSH_ENABLED=false` origin, `RTC_TIER_MAX_VIEWERS=3`, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q102. Voice AI while microphone muted?

PART 12 - Demo Troubleshooting

Publisher track muted - ASR receives silence; agent prompts user to unmute. Client should surface mute indicator tied to ai RTC track state.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and deliveryTier.ts handle scale, RTM sharding in lifecycle.ts handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with requiresLiveSession remains enforceable because eligibility.ts evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (MEDIA_PUSH_ENABLED=false origin, RTC_TIER_MAX_VIEWERS=3, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q103. End session while user in checkout?

PART 12 - Demo Troubleshooting

lifecycle end reprices cart removing LIVE20; checkout page refetches via cart.updated SSE; user sees updated total before pay.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and deliveryTier.ts handle scale, RTM sharding in lifecycle.ts handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with requiresLiveSession remains enforceable because eligibility.ts evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (MEDIA_PUSH_ENABLED=false origin, RTC_TIER_MAX_VIEWERS=3, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q104. Same user in two browser tabs?

PART 12 - Demo Troubleshooting

Same session cookie shares cart; dual voice conversations may conflict on ai:agents lease - second start may 503; RTM same user-<userId> login ref counted in RtmProvider.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and deliveryTier.ts handle scale, RTM sharding in lifecycle.ts handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with requiresLiveSession remains enforceable because eligibility.ts evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (MEDIA_PUSH_ENABLED=false origin, RTC_TIER_MAX_VIEWERS=3, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q105. PUBLIC_API_URL/ngrok misconfigured - diagnose?

PART 12 - Demo Troubleshooting

ConvoAI join succeeds but tool calls fail timeout; check PUBLIC_API_URL reachable from public internet, /mcp returns 401 without signature not connection refused; nginx sticky headers.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and deliveryTier.ts handle scale, RTM sharding in lifecycle.ts handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with requiresLiveSession remains enforceable because eligibility.ts evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (MEDIA_PUSH_ENABLED=false origin, RTC_TIER_MAX_VIEWERS=3, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.

Q106. ConvoAI join returns 429?

PART 12 - Demo Troubleshooting

Agora quota or rate limit; reduce concurrent agents; retry with backoff; fall back LLM_PROVIDER text; verify project ConvoAI entitlement and billing.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone. This pattern keeps the live experience, private ConvoAI conversation, and shopping cart coherent even when RTC, RTM, or MCP experience transient faults.

Operationally, we treat this as a production-grade boundary: Agora never holds catalog or promotion rules; every price, stock check, and eligibility decision flows through packages/domain-commerce and resolveLineContext in eligibility.ts, which joins product_variants, live_sessions, and live_session_products in one SQL proof at evaluation time. State changes propagate via Redis channels events:user, events:session, and events:global to the SSE gateway, where clients refetch authoritative React Query snapshots rather than trusting event payloads alone.

From a Director of Solutions Architecture perspective, the prototype demonstrates separation of concerns that survives requirement changes: Media Push and deliveryTier.ts handle scale, RTM sharding in lifecycle.ts handles chat fan-out, ConvoAI plus MCP handles private voice commerce, and LIVE20 with requiresLiveSession remains enforceable because eligibility.ts evaluates truth at cart-read time rather than trusting browser or LLM state.

For interview demos and production hardening alike, we document honestly what is simulated (MEDIA_PUSH_ENABLED=false origin, RTC_TIER_MAX_VIEWERS=3, mocked payments) versus what is fully implemented (promotion engine, MCP HMAC auth, tool idempotency, session lifecycle side effects, Redis presence, SSE repricing on session end). That transparency builds customer trust and guides the next sprint priorities.